

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Ingeniería de Computadores

Identificación de voces e instrumentos en una obra musical

Martín Quevedo Poyal

Dirección

Basilio Sierra Araujo
Izaro Goienetxea Urkizu

20 de julio de 2025

Resumen

En este Trabajo de Fin de Grado se ha desarrollado, junto con una versión propia de uno de estos modelos, un entorno para entrenar modelos de identificación y separación de elementos en una obra musical. Para ello se ha implementado un modelo de inferencia de máscaras entrenado con bases de datos proporcionadas, añadiendo una función de pérdida basada en *Deep Feature* que implementa la distancia *Earth's Mover Distance* (EMD). Posteriormente se ha evaluado y comparado el rendimiento del modelo al de otros ya publicados en el *State of Art* y su capacidad para separar e identificar elementos musicales en una pista.

Una vez entrenado el modelo, se ha implementado una interfaz que permite aplicarlo junto con la posibilidad de aplicar también sobre una pista el primer modelo inicial en el que se basa el desarrollado en este trabajo junto con una tercera opción en la que se permite la configuración y aplicación de un modelo parametrizado por el usuario.

Esta interfaz permite al usuario hacer uso de modelos previamente entrenados con variedad de funciones de pérdida junto con información pertinente a esta, y permite también la personalización, entrenamiento, y aplicación de un modelo propio, acompañando al proceso con datos sobre los parámetros y terminando con comparativas entre modelos y evaluación y comparación del desempeño de estos para la tarea objetivo con el de otros previamente entrenados.

ODS

Los Objetivos de Desarrollo Sostenible (*ODS*), establecidos por las Naciones Unidas, constituyen un marco global que busca abordar los grandes desafíos sociales, económicos y medioambientales que enfrenta la humanidad. La investigación y el desarrollo tecnológico, especialmente en campos como la inteligencia artificial y el aprendizaje automático, pueden desempeñar un papel clave en la consecución de estos objetivos.

Este proyecto, centrado en el desarrollo y entrenamiento de modelos de aprendizaje automático con un objetivo concreto, siendo este la identificación y separación de elementos en una obra musical, se alinea con varios de los Objetivos de Desarrollo Sostenible establecidos por las Naciones Unidas. A continuación, se ponen en detalle los objetivos más relevantes con los que este proyecto presenta simpatía y se comenta la aportación de este a cada uno de ellos.

ODS 4: Educación de calidad

Este *ODS* tiene como objetivo garantizar una educación inclusiva, equitativa y de calidad y promover oportunidades de aprendizaje durante toda la vida para todos.

El sistema presentado en este proyecto, promueve oportunidades de aprendizaje para todos, identificando las funcionalidades más importantes de un modelo de aprendizaje automático, ilustrando las aplicaciones de estos en un ejemplo del mundo real, como puede ser la identificación de elementos en una obra musical y finalmente presentándolas al usuario de forma interactiva y transversal, con explicaciones por parámetro y método que permiten tanto a usuarios experimentados como a primerizos la interacción e introducción a esta tecnología que cada día está más presente en nuestras vidas.

ODS 3: Salud y bienestar

Este *ODS* busca garantizar una vida sana y promover el bienestar para todos en todas las edades.

Si bien es sabido que el baile (y el disfrute de la música) es una actividad que , como ya se ha demostrado científicamente por multitud de estudios, promueve una mejora de la salud física y emocional del individuo, la separación de fuentes de una obra es ampliamente utilizada para estos propósitos, bien por por *DJs* , *Beatmakers* o *Samplers* mediante diferentes técnicas dentro de estos ámbitos. Si bien esto e hacía antiguamente con métodos más rudimentarios , la introducción de la inteligencia artificial y en concreto de las redes neuronales artificiales como herramienta en este tipo de métodos , facilita y mejora el proceso , lo que deriva en el resultado de una mejor herramienta para el artista y por ende en disfrute para el público de este.

Además, este tipo de herramientas son ampliamente utilizadas para la práctica *amateur* de instrumentos musicales , bien para obtener la pista musical de la guitarra en una canción y poder replicarla de oído en casa con tu propio instrumento , o bien para obtener la pista vocal y acompañarla con tu propia partitura.

Estas posibilidades que nos da la herramienta contribuyen lateralmente al bienestar emocional, abriendo bien un abanico de posibilidades creativas en casa o bien un abanico de mejoras en la metodología artística de los “artesanos” de estas disciplina.

Índice general

Índice de figuras

Índice de cuadros

CAPÍTULO 1

Introducción



En el siglo 21, la digitalización como mejora de herramientas ya es costumbre en la vida cotidiana, tanto para la persona de a pie como para la actividad profesional en amplio abanico de ámbitos.

Las herramientas y facilidades que puedan utilizar ambos pasan cada día más por procesos de revisión constante en los que se introducen elementos digitales que mejoran sus funciones y facilitan su uso, permitiendo a las personas¹ entender y habitar más y mejor el entorno que les rodea, facilitando también su trabajo y aumentando la información que puedan tener sobre este.

La inteligencia artificial viene desarrollándose en nuestra sociedad desde mediados del siglo pasado, en concreto desde los años cuarenta. [1], teóricamente por el escritor Isaac Asimov como referencia cultural o ética de la robótica "*inteligente*", y técnicamente por el matemático Alan Turing , que ya trabajaba por los mismos años en una máquina para romper criptografías enviadas por los alemanes en la segunda guerra mundial, máquina que rompió con tal capacidad el enigma criptográfico del momento , que dio lugar a estudio presentando el *benchmark* actual conocido como *Turing Test* , en 1950 formuló el "imitation game" en Computing Machinery and Intelligence, que más tarde se popularizó como Test de Turing. Método para identificar la inteligencia en un sistema artificial. Este trabajo se paralizó en el momento por la falta de capacidad computacional tras el intento de simulación de redes neuronales , por el momento, computacionalmente imposible.

Posteriormente, ya en el año 2016 , la capacidad de cómputo de las máquinas mas recientes abre la posibilidad de definir y entrenar redes neuronales , ganando así por primera vez una máquina al campeón mundial del juego de mesa *Go*, hito que marca un antes y un después pues hasta el momento se consideraba impensable por la complejidad del juego. Tras esto , el desarrollo

¹En esta memoria de TFG se va a tratar de hacer un uso no sexista de la lengua castellana , por ello se decide adoptar la denominación "persona" refiriéndose tanto a hombre como a mujer.

de la inteligencia artificial llega hasta el que conocemos hoy día proporcionando reconocimiento facial ,reconocimiento del habla y conducción automática de vehículos motorizados, entre otras muchas aplicaciones.

Dentro del ámbito de la inteligencia artificial , la separación de fuentes musicales es un campo que ha experimentado una evolución desde sus primeros acercamientos en 1973 [2] utilizando un *vocoder homomórfico* para separar la función de excitación y la respuesta al impulso del tracto vocal. Estos métodos se clasifican en tres ramas principales: procesamiento de señales , modulación de audio o discurso y teoría de la probabilidad (acercamientos guiados por datos).

En el procesamiento de señales, se busca un filtrado de la mezcla, ya sea aplicando un filtro de *pasa baja*, *pasa alta* o *de banda* a la mezcla completa , o mediante la aplicación de máscaras en rango $[0-1]$ de ganancias a aplicar a cada elemento en una representación tiempo-frecuencia del audio.

En la modulación de audio/discursos, se estudia la predictibilidad del comportamiento de las señales haciendo diferenciación entre *señales sinusoidales* (o tonos puros) y *ruido* , siendo ambos los extremos en una línea de predictibilidad , por un lado el comportamiento de las señales sinusoidales puede ser predicho en un futuro a partir de muestreos previos , mientras que por otro , el *ruido blanco* se define como una señal impredecible a futuro y su espectrograma presenta energía constante en todo el rango frecuencial. Dentro de esta categoría se presentan varias herramientas , como *cepstrum* , con aplicaciones de estimación de frecuencias fundamentales , derivación de frecuencias cepstrales , y filtrado de señal.

En los acercamientos probabilísticos, se diferencia el *Modelo oculto de Markov* que sin entrar en mucho detalle , es útil para observaciones variantes en el tiempo y el *modelo de mezcla Gaussiano* que da una forma de aproximar una distribución como una suma ponderada de *gaussianos* , útil para crear máscaras.

CAPÍTULO

Planificación

2



2.1. Objetivos

Como bien se ha comentado en la sección anterior , el principal objetivo del proyecto era el desarrollo de una herramienta que identifique y separe los diferentes elementos en una obra musical. Sin ánimo de presentar una herramienta que mejore sustancialmente las posibilidades actuales que presenta el estado del arte, se puso inicialmente el objetivo de la separación de dos fuentes, identificando vocales y acompañamiento , para más tarde en el punto de haber alcanzado el primer objetivo, ampliar a la separación a cuatro fuentes recogiendo vocales , bajo , baterías y el resto del acompañamiento identificado como "*resto*".

2.2. Alcance

El presente proyecto tiene como alcance el diseño, entrenamiento y validación de un modelo de separación de fuentes musicales, orientado a la extracción de *stems* (voz, bajo, batería y acompañamiento).

El trabajo abarca la implementación de distintas funciones de pérdida ($L1$, $L2$, $SI-SDR$, pérdidas en frecuencia, pérdidas de máscara, etc.), el entrenamiento controlado de modelos con diferentes configuraciones de *hiperparámetros*, y la evaluación del rendimiento obtenido a través de métricas de separación.

Adicionalmente, se ha desarrollado una interfaz gráfica muy sencilla que permite cargar audios de prueba y obtener sus *stems* separados, así como almacenar los resultados en forma de archivos de audio y gráficas.

El proyecto no pretende competir con modelos de última generación en separación musical, sino explorar el impacto de diferentes estrategias de entrenamiento y funciones de pérdida en arquitecturas más simples, trabajando con *datasets* limitados y recursos de cómputo reducidos.

2. PLANIFICACIÓN

2.3. Cronograma

2.4. Desviaciones respecto a la planificación inicial

CAPÍTULO

Herramientas utilizadas

3



3.1. Ignite

Librería de alto nivel construida sobre *PyTorch* que facilita el entrenamiento de modelos mediante abstracciones que proporcionan métodos ya implementados para entrenamiento, validación , *checkpoints* ... Además de para generación de métricas, en el proyecto se utiliza principalmente para manejar bucles de entrenamiento/validación, guardado/carga de *checkpoints* y registro de métricas y *logs*.

3.2. Librosa

Librería de *Python* más popular para análisis de audio. Construida sobre *NumPy* , *SciPy* y *Matplotlib*. Utilizada para dar soporte a las transformaciones en dominio tiempo-frecuencia , visualización de ondas y espectrogramas , y cargas de audios.

3.3. Matplotlib

Librería estándar de visualización en *Python*. Proporciona principalmente herramientas para creación de gráficos. En el proyecto se ha utilizado para graficar señales , espectrogramas y métricas generales.

3.4. Nussl

(*Northwestern University Source Separation Library*) , Librería de *Python* especializada en separación de fuentes de audio, proporciona métodos de pre-procesamiento de audio , implementación y comparación de algoritmos de separación de fuentes, modelos pre-entrenados , métricas ... En el proyecto se

3. HERRAMIENTAS UTILIZADAS

ha utilizado para cargas de audio *wav* y *mp3* , obtención y reconstrucción de espectrogramas , construcción del modelo, inferencia de las fuentes ...

3.5. Overleaf

Plataforma online para trabajar con *LaTeX* de manera colaborativa, permite escribir documentos académicos y compartir el archivo permitiendo realizar comentarios a tiempo real sobre este. Se ha utilizado para la redacción de esta memoria de forma que los tutores pudiesen comentar sobre la misma mientras se ha ido redactando.

3.6. Pytorch

Librería de aprendizaje profundo en *Python* desarrollada por Meta (previamente *Facebook*) enfocada en la creación , entrenamiento y despliegue de redes neuronales. Se utiliza en el trabajo para definir el modelo de *MaskInference*, trabajar con los tensores, con las funciones de pérdida, para proporcionar métodos de entrenamiento y *backpropagation*, gestión del uso de CPU y carga y guardado de *checkpoints*.

3.7. Streamlit

Librería de *Python* pensada para creación de interfaces gráficas y aplicaciones *web* interactivas de manera simple y enfocada a ciencia de datos e IA. En el proyecto proporciona la implementación de lo que sería la parte del *Frontend* del proyecto.

3.8. Torchaudio

Librería oficial de *Python* centrada en el procesamiento de audio y voz proporcionando *datasets*, transformaciones, y modelos pre-entrenados. Se utiliza en el proyecto para carga de audios, transformaciones, *Data Augmentation*, acceso al dataset *MUSDB...*

CAPÍTULO Marco teórico 4



4.1. Sonidos y señales

Naturaleza del sonido

En esta sección se hablará de la naturaleza del sonido. El sonido es una perturbación mecánica que se propaga en medios elásticos (aire, agua o sólidos) y que percibimos a través del sistema auditivo.

Ondas sonoras

En esta subsección se comentará la naturaleza física del sonido.

Las ondas sonoras son oscilaciones longitudinales que provocan variaciones en la presión del medio, zonas más densas (compresiones) y zonas menos densas (rarefacciones). Se generan por una fuente vibratoria, como una cuerda, un altavoz, cuerdas bucales o una gota de agua y esa vibración se transmite por colisión entre partículas del medio. [3]

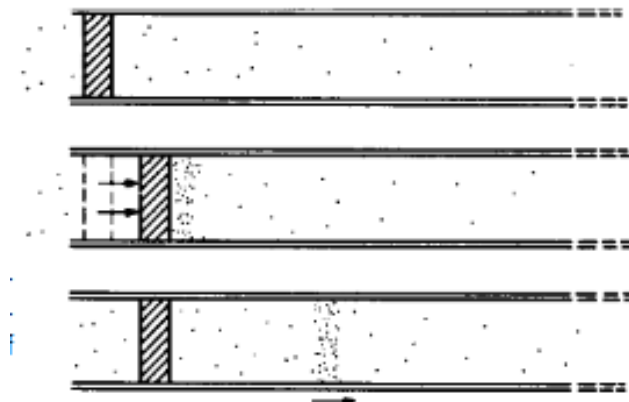


Figura 4.1: Generación de una onda sonora longitudinal mediante el movimiento de un pistón al final de un tubo [3].

Frecuencia y tono

La frecuencia es el número de ciclos completos de la onda por segundo, medida en hercios (Hz) y determina el tono de lo que escuchamos. Las notas tienen sus propias frecuencias, por ejemplo $440Hz$ corresponde a $La4$, la voz humana suele ir en el rango $128-256\text{ Hz}$, los ultrasonidos se consideran los superiores a 20 kHz y los infrasonidos los inferiores a 20 Hz [4].

Por otro lado, el tono está relacionado con la *frecuencia fundamental* (aunque en sonidos complejos intervienen también armónicos, que son múltiplos de la frecuencia fundamental, lo cual afecta también al timbre, siendo el timbre lo que nos permite distinguir una misma nota tocada en una guitarra, un violín, o una flauta.)

Amplitud e intensidad

La amplitud corresponde al máximo desplazamiento de las partículas en el medio respecto a su posición de equilibrio, reflejándose en el sonido como la magnitud de variación de presión o vibración.

Se percibe como *volumen o sonoridad*, ondas con mayor amplitud suenan más fuertes.

La intensidad sonora mide la potencia transmitida por unidad de área (W/m^2) en la dirección del sonido y está relacionada con la amplitud al cuadrado[5]:

$$I \propto A^2$$

Dominio tiempo frecuencia

En esta sección se clarificarán conceptos que se han tenido que estudiar para llevar a cabo el proyecto y relacionados con el dominio tiempo-frecuencia, como magnitud y fase, señales mono y estéreo, transformadas de Fourier y espectrogramas.

Magnitud y fase

La magnitud y fase son los dos principales componentes sonoros en los que descomponemos una señal. Se obtienen mediante la aplicación de la *transformada en tiempo corto de Fourier*. La magnitud indica la energía de la señal en una frecuencia y tiempo dados:

$$|X(t, f)|$$

Mientras que la fase determina el desfase temporal relativo de esa componente:

$$\angle X(t, f)$$

Aunque la magnitud es frecuentemente usada como entrada a modelos de separación, la fase es esencial para reconstruir una señal con facilidad, y también la componente que más complicaciones supone, pues no siempre puede asumirse uniforme y se suele reutilizar la fase de la mezcla como aproximación[6].

Señales mono y estéreo

Son los dos tipos de señales sonoras. Las señales mono se presentan en un solo canal sin información espacial, mientras que las estéreo presentan dos canales, izquierdo y derecho, por lo que crean una sensación espacial y de amplitud dinámica.

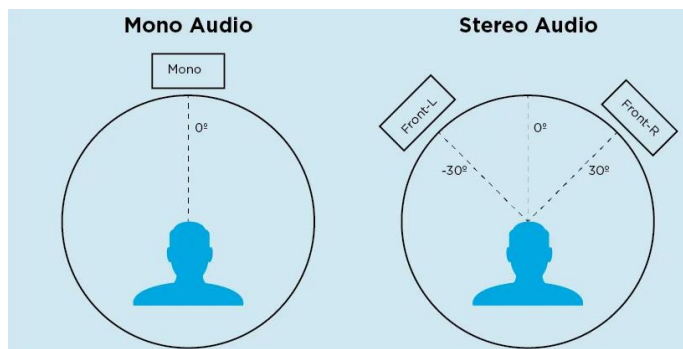


Figura 4.2: Diferencia visual en una habitación entre un sonido mono y uno estéreo.

STFTs

Se calcula sobre la representación en forma de onda del sonido computando una transformación discreta de Fourier (una transformación discreta de Fourier consiste en la captura mediante ventanas triangulares en movimiento L de secciones de la onda) trasladada a lo largo de la onda (L ,2L, 3L ...)

$$X(n, \omega) = \sum_{m=-\infty}^{\infty} x[m] \cdot w[m - n] \cdot e^{-j\omega m}$$

Cada celda (ventana) de la transformada es compleja (contiene magnitud y fase) y la transformada en general es invertible, por lo que se puede obtener la onda inicial a partir de la representación.

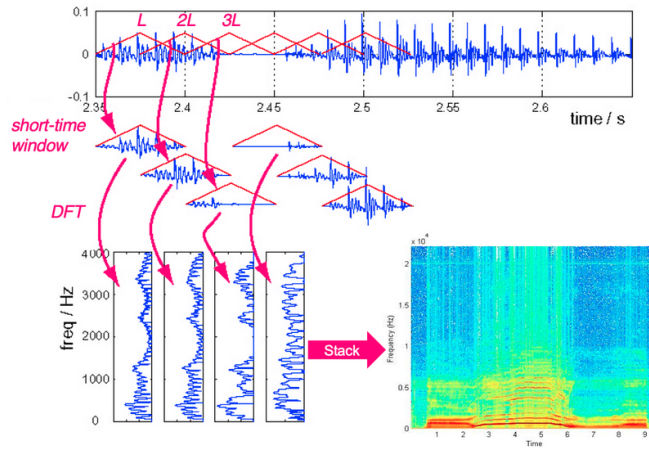


Figura 4.3: Proceso de computar una transformada en tiempo corto de Fourier a partir de una onda. Imagen de Bryan Pardo

[2]

Como parámetros importantes, presenta el tipo de ventana que utiliza (gaussiana, rectangular , triangular, cuadrática ...), la longitud de la ventana y la distancia entre ventanas adyacentes.

Espectrogramas

Es la representación visual de la magnitud de la STFT, y contrapone gráficamente tiempo y frecuencia haciendo una gradiente de color a mas oscuro cuanto mas intensidad/magnitud presente la señal en ese punto.

Existen diferentes tipos, en función de cómo se traten los respectivos elementos de la STFT (magnitud , cuadrado, logarítmico, logarítmico cuadrado)

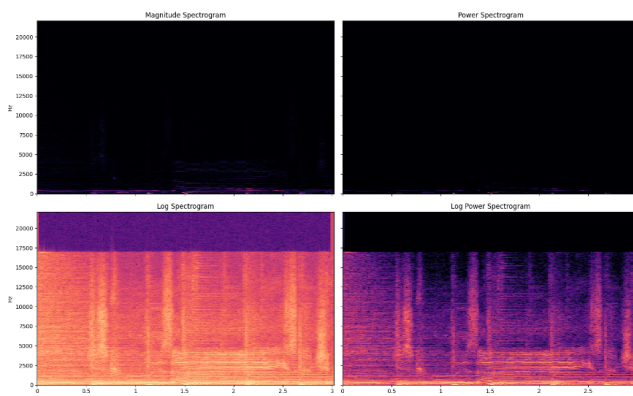


Figura 4.4: Diferentes generaciones de espectrogramas según el tratamiento de la STFT.

[2]

Mezcla de señales y separación de fuentes

En esta subsección se comentarán las diferentes metodologías estudiadas durante la realización de este proyecto en torno a la mezcla de señales sonoras.

Mezclas lineales

Una mezcla lineal de señales se produce cuando varias fuentes

$$s_i(t)$$

se suman con diferentes pesos o coeficientes:

$$x(t) = \sum_{i=1}^N a_i \cdot s_i(t)$$

En música esto ocurre cuando diferentes instrumentos o voces se graban a través de micrófonos cada uno con diferente ganancia, esto simplifica la realidad física de como el sonido se propaga y se capta y ha facilitado el desarrollo de muchos de los modelos actuales con este objetivo. A comentar, la fórmula que se arriba realmente es correcta con aproximación. En casos reales musicales es mas complicado. La música real incluye paneo, reverberación, efectos, retardos...

Mezclas coherentes e incoherentes

Las mezclas coherentes son aquellas que estan correlacionadas en tiempo y frecuencia, por ejemplo , dos guitarras tocando el mismo *riff* en fase, o un coro cantando al unísono.

Las mezclas incoherentes son aquellas que tienen baja correlación, por ejemplo, una batería y una vocal que ocupan bandas de octavas diferentes en un gráfico frecuencial.

Estas últimas son mas fáciles de separar pues los algoritmos pueden aprovechar esta diferencia frecuencial.

Separación ciega de fuentes

La separación ciega de fuentes es aquella que busca separar las señales originales a partir únicamente de las mezclas observadas, sin información previa sobre las fuentes ni sobre el proceso de mezcla.[7]

Dos técnicas clásicas son:

- ***Independent Component Analysis ICA***: busca componentes estadísticamente independientes, muy usada en aplicaciones de separación de voz.
- ***Non-negative Matrix Factorization NMF***: descompone el espectrograma en bases y activaciones, útil en música.

Representaciones digitales

En esta sección se explicarán las diferentes representaciones digitales del sonido y cómo se han utilizado y tenido en cuenta en el proyecto.

Formatos del audio

Las diferentes maneras de representaciones digitales que existen para el sonido digital no solo son una secuencia de muestras, sino que además se guardan en diferentes formatos:

- **Formatos sin compresión**: *WAV*, *AIFF*. Formatos que guardan directamente las muestras de audio codificadas, ofrecen mayor fidelidad al original pero también generan archivos más pesados. Es el formato utilizado en todo el proyecto.
- **Formatos comprimidos sin pérdida**: *FLAC*, *ALAC*. Formatos que reducen el tamaño sin alterar la información.
- **Formatos comprimidos con pérdida**: *MP3*, *AAC*, *OGG*.... Utilizan modelos psicoacústicos para descartar información que el oído humano no percibe. Se usan en transmisión y almacenamiento masivo de audio para consumo (*streaming*, etc.).

Muestreo y cuantización

Para pasar de un sonido analógico (continuo en tiempo y amplitud) a uno digital, se utilizan dos procesos:

- **Muestreo (*sampling*):** consiste en tomar mediciones del valor de la señal en intervalos regulares de tiempo, la frecuencia de muestreo indica cuántas muestras por segundo se capturan y según el teorema de **Nyquist-Shannon** la frecuencia de muestreo debe ser al menos el doble de la frecuencia máxima que queremos registrar. [8]

Tasa de muestreo

La tasa de muestreo determina la fidelidad y el rango de frecuencias que puede representarse:

- **44.1 kHz:** estándar para *CDs*. Permite cubrir hasta 22.05 kHz, que está por encima del rango auditivo humano.
- **48 kHz:** usado en producción audiovisual, audio para video y transmisión digital.
- **96 kHz y 192 kHz:** considerados *alta resolución*, aunque la diferencia es mínima para el oyente, se utilizan en investigación y procesamiento para evitar *aliasing* o realizar transformaciones más detalladas.

4.2. Aprendizaje automático

Conceptos fundamentales

En esta sección se comentarán conceptos relativos al marco teórico del segundo ámbito de la ingeniería tocado en este proyecto, el aprendizaje automático. Se comentarán conceptos fundamentales generales, conceptos fundamentales sobre las redes neuronales artificiales y sobre las técnicas de evaluación y métricas utilizadas en el proyecto. Finalmente se comentará el estado del arte actual en lo relativo al objetivo del trabajo.

Aprendizaje supervisado vs no supervisado

El aprendizaje supervisado y no supervisado son dos enfoques fundamentales en el campo del aprendizaje automático, cada uno con sus propias características, aplicaciones y desafíos. El aprendizaje supervisado se basa en la utilización de datos etiquetados para entrenar modelos que puedan hacer predicciones o clasificaciones precisas. En contraste, el aprendizaje no supervisado se enfoca en descubrir patrones y estructuras ocultas en datos no etiquetados, lo que lo hace especialmente útil para tareas como la agrupación y la reducción de dimensionalidad.

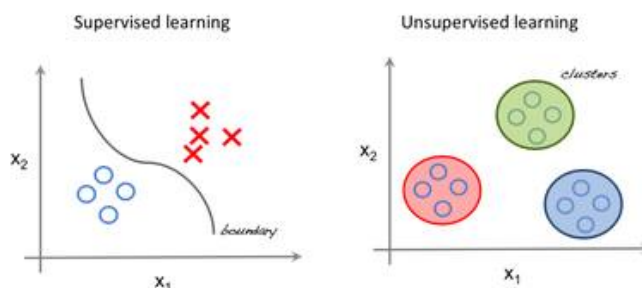


Figura 4.5: Comparativa de clasificación entre aprendizaje supervisado y no supervisado. Imagen de Juan Domingo Farnos [2]

Overfitting, underfitting y generalización

Otros dos conceptos importantes a la hora de trabajar con algoritmos de inferencia de clases, son el overfitting y el underfitting. El overfitting ocurre cuando un modelo se ajusta demasiado a los datos de entrenamiento, capturando ruido o patrones específicos que no generalizan bien a nuevos datos. Esto puede resultar en un rendimiento excelente en el conjunto de entrenamiento pero pobre en datos no vistos. Por otro lado, el underfitting sucede para el caso contrario, cuando un modelo es demasiado simple para capturar la complejidad de los datos, lo que lleva a un rendimiento deficiente tanto en el conjunto de entrenamiento como en datos nuevos. Finalmente otro concepto importante es la generalización. Esta se refiere a la capacidad de un modelo para desempeñarse bien en datos no vistos, y es crucial encontrar un equilibrio entre overfitting y underfitting para lograr una buena generalización.

Dataset

El concepto de dataset refiere al conjunto de datos intencionalmente elegidos para ser utilizados en el entrenamiento, validación o prueba de un modelo de aprendizaje automático. Un dataset bien diseñado es crucial para el éxito de cualquier proyecto de aprendizaje automático, ya que la calidad y representatividad de los datos pueden influir significativamente en el rendimiento del modelo. A modo de ejemplo, en el contexto de este proyecto se han utilizado dos datasets diferentes para los procesos de entrenamiento-validación y evaluación. La intención detrás de esta elección es garantizar que el modelo se entrene y valide con datos que reflejen una amplia variedad de casos, mientras que la evaluación se realiza con un conjunto de datos completamente independiente para medir la capacidad de generalización del modelo. Esta estrategia ayuda a evitar problemas como el overfitting, donde el modelo se ajusta demasiado a los datos de entrenamiento y no generaliza bien a nuevos datos.

RNNs artificiales

Tipos de redes

Sin entrar mucho en detalle, aquí se mencionan las arquitecturas de redes neuronales más comunes, cada una con sus propias características y aplicaciones específicas.

- **Redes Neuronales Feedforward (FNNs):** Son las redes neuronales más simples, donde la información fluye en una sola dirección, desde las capas de entrada hasta las capas de salida, sin ciclos ni conexiones recurrentes. Son adecuadas para tareas de clasificación y regresión.
- **Redes Neuronales Convolucionales (CNNs):** Están diseñadas para procesar datos con una estructura de cuadrícula, como imágenes. Utilizan capas convolucionales para extraer características espaciales y son ampliamente utilizadas en visión por computadora.
- **Redes Neuronales Recurrentes (RNNs):** Son adecuadas para procesar secuencias de datos, como texto o series temporales. Tienen conexiones recurrentes que permiten mantener información a lo largo del tiempo, lo que las hace ideales para tareas como el procesamiento del lenguaje natural y la predicción de secuencias.

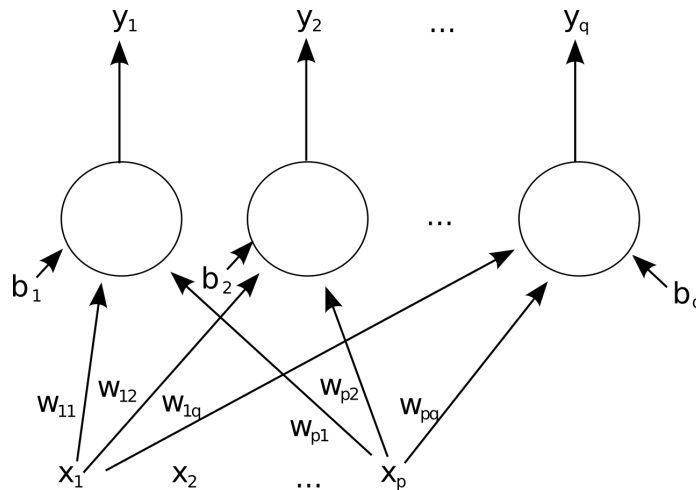


Figura 4.6: Una sola capa de red neuronal artificial feed-forward. Por Mcstrother - Trabajo propio, CC BY 3.0, <https://commons.wikimedia.org/w/index.php?curid=9515940>

[2]

Optimización y descenso de gradiente

Para mejorar una red neuronal, nuestro objetivo es encontrar pesos y sesgos que minimicen la función de coste. Como las redes neuronales suelen tener un gran número de pesos y sesgos, suele tratarse de un problema de optimización de gran dimensión. Para minimizar la función de coste, comentamos a raíz de que es el método utilizado en el proyecto el descenso de gradiente. El descenso de gradiente es un algoritmo de optimización iterativo que se utiliza para encontrar los valores de los parámetros (pesos y sesgos) que minimizan una función de coste. El algoritmo funciona calculando el gradiente de la función de coste con respecto a los parámetros y luego actualizando los parámetros en la dirección opuesta al gradiente, lo que conduce a una disminución de la función de coste. Este proceso se repite hasta que se alcanza un mínimo local o global de la función de coste, lo que indica que se han encontrado los mejores parámetros para el modelo.

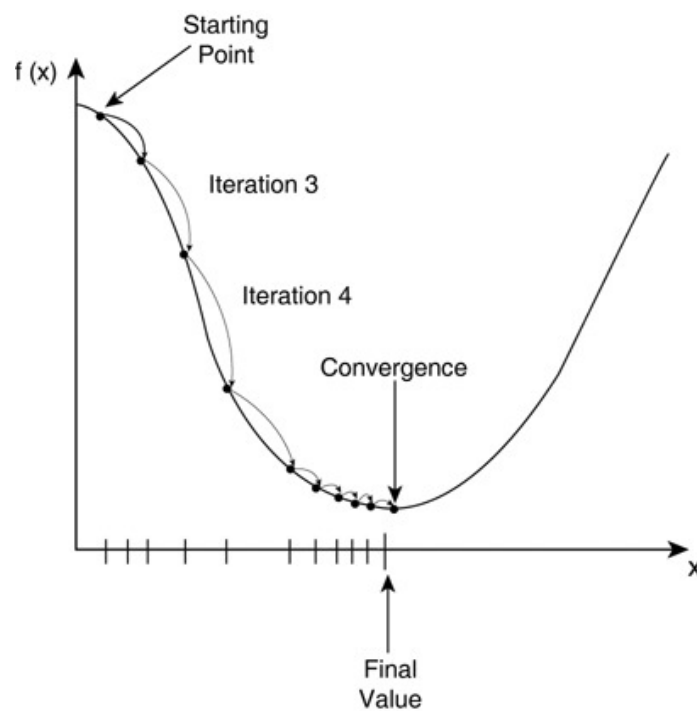


Figura 4.7: Ilustración del proceso de descenso de gradiente.
[2]

Neuronas, capas y funciones de activación y pérdida

Otros conceptos a tener en cuenta:

- Neuronas: unidades básicas de procesamiento que reciben entradas, aplican una función de activación y generan una salida.
- Capas: agrupaciones de neuronas que pueden realizar diferentes transformaciones en sus entradas.
- Funciones de activación: funciones matemáticas que determinan si una neurona debe activarse o no, introduciendo no linealidad en la red.

Técnicas de evaluación y métricas utilizadas

Funciones de pérdida

Data augmentation

Métricas de evaluación

Las técnicas de evaluación , como se comentó en la sección de los datasets son relevantes para verificar las capacidades del modelo y de las características que han definido el proceso de entrenamiento de este. En este sentido, se suelen utilizar métricas que indican estas capacidades, como por ejemplo la precisión, la exactitud, el recall o la F1-score. Estas métricas permiten evaluar el rendimiento del modelo en términos de capacidad para clasificar correctamente los datos, identificar verdaderos positivos y negativos, y equilibrar la precisión y el recall. Además, es importante considerar la matriz de confusión, que proporciona una representación visual de los resultados de clasificación, mostrando las verdaderas clases frente a las predichas por el modelo. Esto ayuda a identificar patrones de error y áreas donde el modelo puede estar fallando.

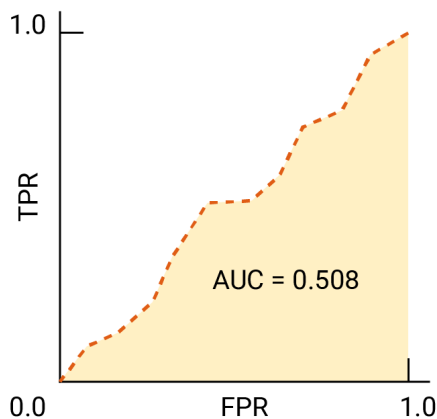


Figura 4.8: Ejemplo de graficación de la métrica AUC. Esta métrica parte de la matriz de confusión generando un gráfico de puntos donde se evalúa la capacidad del modelo para clasificar correctamente en función de los valores por debajo de la diagonal del gráfico.

[2]

Estado del arte

A día de hoy se han hecho progresos significativos en el campo de las redes neuronales artificiales, siendo el principal tema de actualidad en 2026 tenemos un mercado competitivo en el ámbito de los modelos de lenguaje.

Con empresas como *OpenAI* , *Google* , *Meta* o *Microsoft* , entre otras, compitiendo por desarrollar modelos cada vez más avanzados y eficientes que se utilizan en una amplia variedad de aplicaciones, desde chatbots y asistentes virtuales hasta sistemas de recomendación y análisis de sentimientos. Además, se han realizado avances en la comprensión y mitigación de los sesgos en los modelos de lenguaje, así como en la mejora de su capacidad para generar texto coherente y relevante. En resumen, el estado del arte en redes neuronales artificiales es un campo dinámico y en constante evolución, con un enfoque particular en el desarrollo de modelos de lenguaje cada vez más sofisticados y aplicables a una amplia gama de tareas. Tanto es el desarrollo actual que presenciamos como cambian incluso sectores laborales enteros, poniendo como ejemplo la salida más común de mis estudios, el desarrollador de software, actualmente pasa un momento delicado pues cada vez más empresas optan por utilizar modelos de lenguaje para generar código, lo que ha llevado , aunque aún no exista una sustitución de puestos de trabajo, sí que cada vez mas profesionales comentan que llevan meses sin tocar una línea de código y existen casos de despidos masivos de grandes empresas por sustitución de trabajadores

por agentes.

Sin embargo, también se están creando nuevas oportunidades laborales. Se ven ahora nuevas áreas como la supervisión y mejora de los modelos de lenguaje, así como en la investigación y desarrollo de nuevas aplicaciones para esta tecnología.

En general, el estado del arte en redes neuronales artificiales es un campo emocionante y prometedor, con un impacto significativo en la forma en que interactuamos con la tecnología y el mundo que nos rodea.

CAPÍTULO 5

Parte 1 : separación de fuentes

5.1. Diseño del modelo

Tras un estudio en cierta profundidad de los métodos comunmente utilizados para el objetivo que nos concierne ¡Se llega a la conclusión individual de que existen gran cantidad de métodos diversos! [9]

En nuestro caso, y aprovechando el recurso en formato tutorial que presentan los autores anteriormente citados en esta sección, y tras una prueba inicial de generación de máscaras, se elige el modelo utilizado previamente por los autores cuyo funcionamiento apunta a la inferencia de máscaras binarias para la separación de las fuentes.

A tener en cuenta:

- El modelo no separa directamente onda temporal, si no que predice máscaras tiempo-frecuencia.
- La salida estimada es una magnitud que se calcula como máscara de salida por magnitud de la mezcla.
- En el entrenamiento se comparan magnitudes estimadas con magnitudes objetivo.
- En el proceso de inferencia de la máscara se reconstruye el audio usando la fase de la mezcla.

5.2. Arquitectura base

El modelo empleado es una arquitectura recurrente de inferencia de máscaras, basada en una LSTM que recibe la magnitud de la STFT de la mezcla y

predice una máscara por fuente. Estas máscaras se aplican sobre la magnitud de la mezcla para obtener las magnitudes estimadas de cada *stem*.

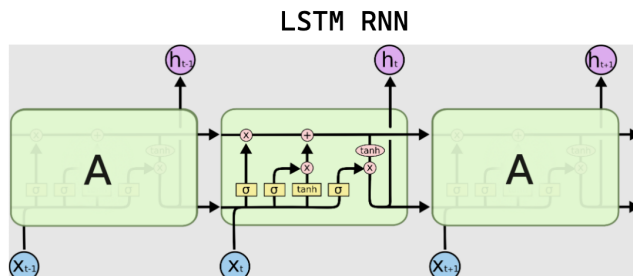


Figura 5.1: Ejemplo de una arquitectura RNN-LSTM
[2]

En la figura se muestra una red neuronal recurrente LSTM desplegada en el tiempo.

Una LSTM procesa una secuencia paso a paso, palabras por ejemplo, sonidos o datos temporales.

Cada bloque verde representa la misma celda LSTM aplicada en diferentes momentos en el tiempo x_t y x_{t+1}

x_t : es la entrada en el instante actual.

h_t : es el estado oculto o salida de la celda en ese instante, lo que la red "entiende" en ese momento

Línea horizontal superior : es el estado oculto o salida de la celda en ese instante. Resume lo que la red ha *entendido* hasta el momento.

Bloque central, contenidos :

- Puerta de olvido: decide qué información antigua se elimina de la memoria.
- Puerta de entrada: decide qué información nueva se guarda.
- Candidato de memoria: genera nueva información posible para añadir a la memoria.
- Puerta de salida: decide qué parte de la memoria se usa para producir h_t

Los símbolos **sigmoid**, **tanh** son funciones que controlan la cantidad que pasa de una información concreta. Por ejemplo, **sigmoide**, produce valores continuos en (0,1) y hace la función de una "perilla" o "compuerta suave" para la información.

5.3. Modificaciones propuestas

Arquitectura y parámetros

En cuanto a proponer un nuevo enfoque a un problema mediante inteligencia artificial, generalmente buena parte de los métodos tratan de aportar alguna configuración de parámetros o modificación a la arquitectura que resulten en unos mejores resultados.

- **Arquitectura:**

La arquitectura base no fue sustituida por un modelo diferente, sino que se estabilizó su implementación. Durante la fase de depuración se detectó que la ruta de inferencia debía coincidir exactamente con la utilizada durante el entrenamiento. Por ello, se mantuvo la estructura basada en inferencia de máscaras, restaurando el preprocesado mediante conversión a decibelios y normalización por lotes antes de la red recurrente. Esta corrección permitió que la LSTM recibiera una representación coherente con la usada durante el entrenamiento, siendo la siguiente:

STFT $\rightarrow$ magnitud $\rightarrow$ AmplitudeToDB $\rightarrow$ BatchNorm $\rightarrow$ LSTM bidireccional $\rightarrow$ Embedding $\rightarrow$ máscaras $\rightarrow$ iSTFT

- **Parámetros:**

Para los parámetros, se tuvo que realizar una segunda versión de estos, ya que, tras considerar una tanda de entrenamientos realizada como válida, al evaluar se llegó a la conclusión de que los parámetros del modelo base y los parámetros de nuestros entrenamientos no coincidían, lo que llevó a una mala generalización del modelo. Por ello, se decidió realizar una segunda tanda de entrenamientos con los parámetros corregidos, que se muestran a continuación:

Parámetro	Valor
Dominio de trabajo	Tiempo-frecuencia, mediante STFT
Tamaño de ventana STFT	512 muestras
Salto STFT	128 muestras
Tipo de ventana	sqrt_hann
Número de bins de frecuencia	257
Canales de entrada	1, mono
Tipo de red recurrente	LSTM
Número de capas recurrentes	1
Tamaño oculto	50
Bidireccional	Sí
Dropout	0.0
Activación de salida	Sigmoid
Salidas en 2 stems	vocals, accompaniment
Salidas en 4 stems	vocals, bass, drums, other

Cuadro 5.1: Configuración arquitectónica base empleada en los entrenamientos V2.

Función de pérdida: *Deep-feature-EMD*

La función de pérdida *Deep-feature-EMD*, propuesta por el tutor de este proyecto donde se hace uso de una distancia tipo *Sinkhorn/EMD* sobre las características extraídas. Define la parte mas experimental del proyecto.

La pérdida *DeepFeature-EMD* compara la estimación y el objetivo no directamente en el dominio de magnitud, sino en el espacio de activaciones de una red convolucional auxiliar. Para cada capa considerada, las activaciones se aplanan y se interpretan como distribuciones[10]. La discrepancia entre ambas distribuciones se mide mediante una distancia *Sinkhorn*, de la *Earth Mover's Distance*, que es aquí donde recibe la titularidad de experimental y también la de propuesta del tutor. La pérdida transforma estimación y objetivo en representaciones espectrales, extrae activaciones mediante una CNN auxiliar y compara las distribuciones de activaciones usando una distancia *Sinkhorn*, aproximación regularizada de *EMD*.

El extractor no está entrenado en música, por lo que la pérdida se considera experimental y no necesariamente correlaciona con calidad perceptual.

5.4. Configuración de entrenamiento

Inputs y targets

La entrada principal al modelo es la magnitud de la mezcla en dominio STFT, en particular:

$$STFT\ compleja + magnitud + tensor$$

Internamente se adapta al formato $[B, T, F]$, se convierte a dB y se normaliza antes de entrar a la RNN. La magnitud lineal se conserva para aplicar la máscara al final.

Los targets (objetivos) son las magnitudes de las fuentes reales. Existen dos modalidades de salida:

- **4 fuentes:** *vocales + batería + bajo + resto*
- **2 fuentes:** *vocales , acompañamiento*

La salida del modelo no genera audio directamente. Aunque en la ejecución lo parezca, la salida son máscaras tiempo-frecuencia. Posteriormente, se aplica la máscara sobre la magnitud de la mezcla y el resultado corresponde a la estimación de la fuente. Luego para reconstruir el audio se utiliza la fase de la mezcla. Esto es una limitación importante, ya que es una estimación. La fase de la mezcla original no tiene porque ser la de las fuentes, pero a falta del dato de la magnitud para estas últimas, se utiliza la de la mezcla.

***TFMs* aplicadas**

En esta sección se comentarán las transformaciones aplicadas al audio/tensores antes del entrenamiento:

- **Transformaciones espectrales** , incluye:
 - STFT.
 - obtención de magnitud.
 - conservación de fase para reconstrucción.
 - posible agrupación de fuentes para 2 stems.
- **Transformaciones internas del modelo** , incluye:
 - Amplitud a decibelios.
 - Normalización del *batch*.
 - Reordenación de dimensiones a $[B, T, F]$.
- **Transformaciones para pérdidas concretas** , incluye:
 - Pérdidas de frecuencia: reformulación a $[B^*S, F, T]$
 - Pérdidas de *deep-feature*: reformulación a $[B^*S, 1, F, T]$

Data augmentation

Para mejorar el desempeño, evitar *overfitting* y mejorar generalización en el modelo.

Se planteaba inicialmente aplicar transformaciones como:

- Reverberación
- Shiftteo de pitch
- Ecualización
- Ruido

Finalmente no se utilizó por problemas entre versiones de librerías. Se menciona aquí una visión positiva de esta complicación, pues así se permite mantener una comparación controlada entre funciones de pérdida.

Preprocesado de un *dataset*

En esta sección se explicará el camino desde los archivos de audio hasta los *batches* de entrenamiento.

- **1. Organización inicial del dataset:** El dataset se compone de mezclas y fuentes separadas. Para cada pista se tiene:
 - *mixture.wav*: la mezcla original.
 - *vocals.wav*: pista para las vocales.
 - *drums.wav*: pista para la batería.
 - *bass.wav*: pista para el bajo.
 - *others.wav*: pista para elementos extra.
- **2. Construcción de escenarios:** Como ya se ha comentado en secciones anteriores, el modelo contempla dos escenarios de salida: cuatro y dos pistas. Partiendo de ello, el dataset de entrada para los dos tipos de entrenamiento debe ser acorde a la salida. Para ello:
 - *2 stems*: se mantienen las vocales y se agrupa el resto como acompañamiento.
 - *4 stems*: se usan las cuatro fuentes originales.
- **3. Conversión a dominio espectral:** Se calcula la STFT de mezcla y fuentes con los mismos parámetros. Se extraen magnitudes para entrenamiento y se conserva la fase de la mezcla para reconstrucción/evaluación.
- **4. Preparación de *batches*:** Cada *batch* contiene al menos:

- *mix_magnitude*
- *source_magnitudes*

Y para pérdidas que piden fase:

- *mixture_phase*
- *source_phase*

La versión final del proceso de entrenamiento comprueba la presencia de estos dos últimos para las funciones que las piden, y usa **source_magnitudes** como *target* principal durante el entrenamiento.

■ **5. División train/validation/test:**

- *train/validation*: usado durante entrenamiento y *early stopping*.
- *representative_test*: conjunto fijo usado solo para evaluación final.

División 60-40.

- **6. Normalización/canales:** El sistema se configuró para un canal de audio, lo que reduce el coste computacional y se recomienda en la documentación del modelo[9]. También simplifica el proceso de comparación entre fuentes.

5.5. Entrenamiento del modelo

Esta sección explicará bajo qué lógica se entrenó, el qué se entrenó, qué observaciones salieron durante el proceso y responderá a las preguntas:

- ¿Qué experimentos se entrenaron?
- ¿Por qué se organizaron así?
- ¿Cómo ha sido la evolución experimental?

Resultados preliminares y experimentales

Los entrenamientos fueron agrupados por funciones de pérdida, en particular por una estructura de tipos dentro de estas para ayudar a su posterior documentación y evaluación:

- **Pérdidas principales:** *L1*, *L2*, *L1_freq*, $\log L1$, $\log L2$, $\log_mag$, $\log_compressed_l2$, *LPSA*, *mask_L1*.
- **Pérdidas experimentales:** *deep_feature*, *deep_feature_emd*.
- **Pérdidas avanzadas:** *l_mrs*, *lpsa_phase*.

También existe la división que ya se explicó anteriormente entre dos y cuatro fuentes. El caso de las dos fuentes representa una separación más sencilla y cercana a escenarios de voz y acompañamiento. El caso de cuatro fuentes introduce mayor complejidad, ya que el modelo debe reppartir la mezcla entre varias fuentes instrumentales con solapamiento espectral. Se respetó la estructura que presenta el *dataset* utilizado de *MUSDB18*, compuesto por 150 pistas con *stems* aislados, con etiquetas en el orden de *vocales, baterías, bajos y otros* y dividido en conjuntos de entrenamiento y test.

Comparativas

Aquí vamos a hacer una comparativa preliminar del conjunto de funciones presentado junto con sus versiones de dos y cuatro fuentes.

■ A. Comparación por número de fuentes

La documentación utilizada y los recursos citados en este proyecto en múltiples ocasiones comentan las diferencias entre estos dos enfoques al problema. La separación que resulta en el conjunto de dos fuentes, ya visto en las primeras versiones del entrenamiento, es un enfoque más sencillo que permite una "binarización" del problema al reducirse a dos salidas. En contraposición con el segundo conjunto, donde la separación se extiende a identificar cuatro elementos en el espectro, los elementos que se puedan identificar si el método es válido pueden variar enormemente dificultando así la identificación de estos, ya que, un elemento que a lo mejor está en un rango frecuencial concreto en una mezcla, en otra puede habitar en otro no a lo mejor completamente diferente pero sí lo suficiente para que la sensibilidad de la función a estos cambios no dé al modelo la información necesaria para ser capaz de identificar un elemento que varía. Por esto, es esperable que, por lo general, 4 *stems* obtenga peores métricas en la fase de evaluación. Identificar y separar dos elementos en contraposición y por lo comentado, resulta en una tarea más sencilla ya que la variabilidad se reduce a eso, dos elementos, mientras que el caso anterior eleva la premisa de la necesidad de sensibilidad en la función al doble de elementos donde puede variar este rango frecuencial.

Por ejemplo, géneros como la electrónica o el rap que se basan en elementos frecuencialmente bajos (bajo, sub-bajo) presentan gran superposición entre fuentes como bajo y baterías, es un sonido más "sucio" armónicamente complejo. Por otro lado, por ejemplo, el blues, pop, o canciones de autor, presentan mejores desempeños en la separación por sus características de armónicos vocales e instrumentales más limpios [Arxiv].

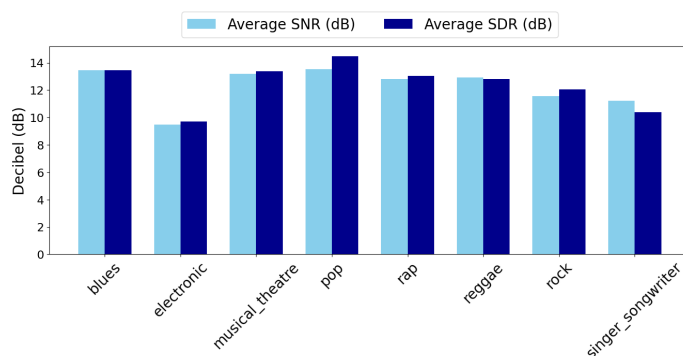


Figura 5.2: Average SNR and SDR by genre

■ B. Comparación por familia de pérdida

Se han agrupado las funciones de pérdida por las familias aritméticas a las que pertenecen por su naturaleza:

- Pérdidas de magnitud directa: L1 , L2.
- Pérdidas frecuenciales: L1_freq, L2_freq.
- Pérdidas logarítmicas: logL1, logL2, log_mag, log_compressed_l2.
- Pérdidas informadas por fase o potencia: LPSA, lpsa_phase.
- Pérdidas basadas en máscara: mask_l1
- Pérdidas experimentales: deep_feature , deep_feature_emd

Las pérdidas logarítmicas buscan reducir el efecto de diferencias absolutas grandes en *bins* energéticos y prestar más atención a estructuras de menor energía. Las frecuenciales, comparan directamente la representación tiempo-frecuencia, mientras que las basadas en máscara fuerzan al modelo a aproximar una asignación explícita de cada bin a una fuente.

Reentrenamiento V2

Esta versión nace como corrección de la primera versión .

La primera versión, como ya se comentaba en el punto anterior, entrenaba con señales estéreo sin haber cerrado todavía el criterio de canales. Esto hacía que los tensores de entrada y salida no estuvieran alineados con la arquitectura final planteada..

V2 reentrena los modelos con una configuración mas coherente, se entrenan modelos de dos y cuatro fuentes de salida, y sirve para obtener una primera batería de seria de *checkpoints*. Durante la primera evaluación de V2 , se detectan nuevos problemas: carga de checkpoints, diferencias entre entrenamiento e

inferencia, stems muy correlacionados con la mezcla y resultados que no superaban *baseline*. Finalmente, y por estos motivos, se termina por considerar el conjunto de modelos entrenados como insuficiente, aunque no se descarta completamente y se deja en el repositorio a modo de marca temporal para mostrar la evolución del proyecto.

Reentrenamiento V3 y protocolo final

El reentrenamiento numero tres corresponde a la versión actual y a la que ha proporcionado los resultados que se muestran en esta memoria para comparar el desempeño de las funciones de pérdida. Nace en respuesta a lo visto y aprendido en la versión anterior.

En esta versión se introduce un conjunto de test fijo, se añade una *baseline* trivial de mezcla y se unifica la inferencia entre evaluación y despliegue , como cambios principales.

Y más en profundidad:

- Se restaura el *forward* coherente con el entrenamiento que se había modificado en la versión anterior a raíz de sospechar que ese era el punto que no permitía una generación de resultados correcta:

AmplitudeToDB + BatchNorm + RNN + Embedding

- Se separan pérdidas principales y experimentales.
- Se organiza la salida en un nuevo directorio.
- Se guarda un sumario de configuración de entrenamiento por modelo. Esto ayuda a hacer un seguimiento a posteriori.
- Se ajusta la planificación por coste final

CAPÍTULO

Parte 2 : interfaz 6



6.1. Objetivos

Interacción y aprendizaje

6.2. Diseño de experiencia de usuario

Organización y visualización

Explicaciones y guías

6.3. Casos de uso

Entrenamiento parametrizable

Parámetros principales ajustables

Visualización de gráficos y espectrogramas

Feedback

Integración del sistema

CAPÍTULO 7



Backend

Comunicación entre módulos

Incidencias de integración y depuración

Evaluación del sistema

CAPÍTULO 8



Métricas utilizadas

Validación del pipeline de evaluación

Resultados obtenidos

Discusión y análisis

Limitaciones y margen de mejora

Conclusiones y trabajo futuro

CAPÍTULO 9



Síntesis del trabajo realizado

Lineas de mejora

Trabajo futuro

CAPÍTULO
Anexos

10



Código relevante
Registro de incidencias técnicas
Configuraciones
Instrucciones posibles
Resultados adicionales

Bibliografía

- [1] Michael Haenlein y Andreas Kaplan. “A Brief History of Artificial Intelligence: On the Past, Present, and Future of Artificial Intelligence”. En: *California Management Review* 61.4 (2019), págs. 5-14. eprint: <https://doi.org/10.1177/0008125619864925>.
- [2] Z. Rafii et al. “An Overview of Lead and Accompaniment Separation in Music”. En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 26.8 (ago. de 2018), págs. 1307-1335.
- [3] Society of Broadcast Engineers. “The Physical Nature of Sound”. En: *SBE Handbook Fundamentals of Audio* (2025). Accedido: 28 de agosto de 2025.
- [4] LibreTexts Physics. “Introduction to Sound”. En: *LibreTexts Physics* (2025). Accedido: 28 de agosto de 2025.
- [5] OpenStax. “Sound Intensity and Sound Level”. En: *OpenStax Physics* (2025). Accedido: 28 de agosto de 2025.
- [6] Source Separation Tutorial. *Phase in source separation*. <https://source-separation.github.io/tutorial/basics/phase.html>. Accedido: 28 de agosto de 2025. 2025.
- [7] Jean-François Cardoso. “Blind Signal Separation: Statistical Principles”. En: *Proceedings of the IEEE* 86.10 (1998), págs. 2009-2025.
- [8] Claude E. Shannon y Warren Weaver. *The Mathematical Theory of Communication*. Originally published in Bell System Technical Journal, 1948. Urbana, IL: University of Illinois Press, 1949.
- [9] Zafar Rafii et al. “An Overview of Lead and Accompaniment Separation in Music”. En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 26.8 (2018), págs. 1307-1335.
- [10] ... Papendra Kumar Yashvi Chandola Jitendra Virmani. “Deep Learning for Chest Radiographs”. En: *Science Direct* ().